MAQPRINT

PROJET DE STAGE

Application Web de Gestion de Calendrier

POUR

Système de Surveillance de Zone de Stockage

Création, consultation et organisation des événements

BTS SIO SISR - 2ème année

Stage 2025

1. Contexte du projet

MAQPRINT est une entreprise d'impression industrielle disposant de zones de stockage contenant du matériel et des consommables de valeur. Des incidents de vol ont été constatés, nécessitant la mise en place d'un système de surveillance permettant d'identifier les personnes accédant à ces zones.

L'entreprise dispose déjà de caméras IP Hikvision installées dans les zones concernées. L'objectif est de créer une application Web de gestion de calendrier afin de pouvoir créer/modifier/supprimer des évènement et d'attribuer les permission à une ou plusieurs personnes dans une salle précise. Puis, de relier cette application aux équipements existants pour créer un système de détection et d'alerting automatisé.

2. Objectifs

2.1 Objectif principal

Développer une application web de gestion de calendrier capable de créer, modifier et supprimer des : évènements, personnes, salles.

2.2 Objectifs pédagogiques

- Comprendre et structurer une application web complète
- Mettre en œuvre des algorithmes de détection de mouvement
- Manipuler une base de données relationnelle
- Développer des fonctionnalités dynamiques en PHP
- Débogage et résolution de problèmes
- Améliorer l'expérience utilisateur (UX)

3. Spécifications techniques

3.1 Environnement de développement

Pour réaliser ce projet, j'ai utilisé :

Composant	Détails
Système d'exploitation	Windows 11
Langage	Python 3.11.4
Bibliothèques	OpenCV, smtplib, datetime, os
IDE	VS Code
Caméra	Hikvision (flux RTSP disponible)

IA	ChatGPT, Gemini

3.2 Fonctionnalités attendues

Détection de mouvement

- Connexion au flux RTSP de la caméra Hikvision
- Analyse en temps réel des images pour détecter les mouvements
- Paramétrage du seuil de sensibilité
- Définition d'une zone d'intérêt (ROI) optionnelle

Capture d'images et vidéos

- Capture de 3 à 5 images successives lors d'une détection
- Enregistrement d'un clip vidéo de 30 à 60 secondes
- Horodatage visible sur les captures
- Stockage organisé par date (ex: /captures/2025-01-06/)

Système d'alerting

- Envoi d'un e-mail automatique lors d'une détection
- Pièces jointes : captures d'écran
- Informations : date, heure, zone concernée
- Anti-spam : délai minimum entre deux alertes (ex: 60 secondes)

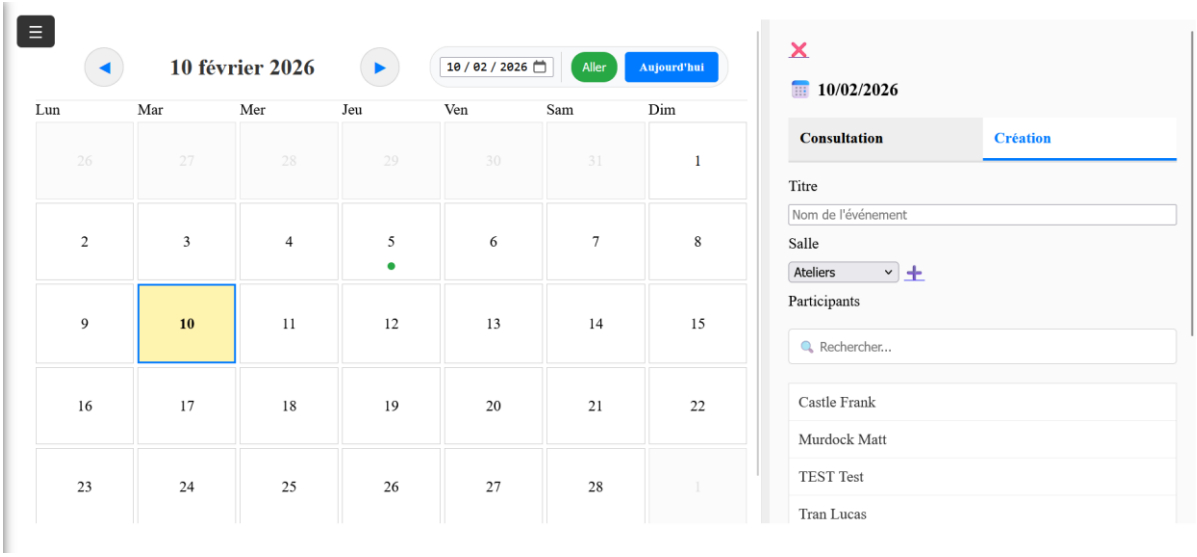
Création du calendrier :

Avant de pouvoir relier les caméras au évènement, il faut dans un premier temps créer le calendrier. Son objectif est multiple :

- Créer des personnes
- Créer des salles
- Créer des évènement
- Modifier/Supprimer des personnes
- Modifier/Supprimer des salles
- Modifier/Supprimer des évènements

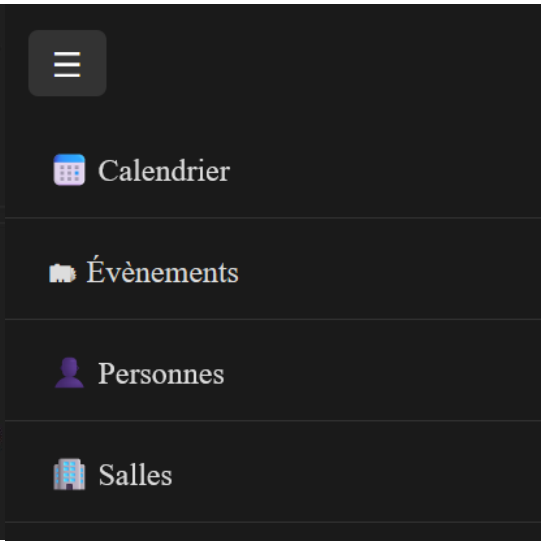
Avant de commencer par quelconque création, il faut créer l'interface du calendrier. Pour cela, je me suis aidé de Gemini. Ainsi, j'ai pu réaliser une interface capable d'avoir les jours d'un mois choisi affichés, de créer/modifier/supprimer autant d'évènements que l'on souhaite pour n'importe quel jour, de naviguer rapidement entre les mois et années.

```php
<div class="calendar-grid">
    <?php foreach (['Lun','Mar','Mer','Jeu','Ven','Sam','Dim'] as $d): ?>
        <div class="day-name"><?= $d ?></div>
    <?php endforeach; ?>

    <?php
    $lastDayOfMonth = strtotime(datetime: "$year-$month-$daysInMonth");
    $endDayOfWeek = date(format: 'N', timestamp: $lastDayOfMonth);
    $endDate = strtotime(datetime: "+".(7 - $endDayOfWeek)." days", baseTimestamp: $lastDayOfMonth);
    $current = $startDate;
    while ($current <= $endDate):
        $dateStr = date(format: 'Y-m-d', timestamp: $current);
        $isCurrentMonth = date(format: 'm', timestamp: $current) == $month;
        $isToday = ($dateStr == date(format: 'Y-m-d'));
        $hasEvent = isset($eventDays[$dateStr]);
    ?>
        <div class="day <?= !$isCurrentMonth ? 'other-month' : '' ?> <?= $isToday ? 'today' : '' ?> <?= $selectedDate == $dateStr ? 'sel
            onclick="location.href='?date=<?= $dateStr ?>&view=day'">
            <span class="day-number"><?= date(format: 'j', timestamp: $current) ?></span>
            <?php if ($hasEvent): ?><span class="event-dot"></span><?php endif; ?>
        </div>
    <?php $current = strtotime(datetime: '+1 day', baseTimestamp: $current); endwhile; ?>
</div>
```



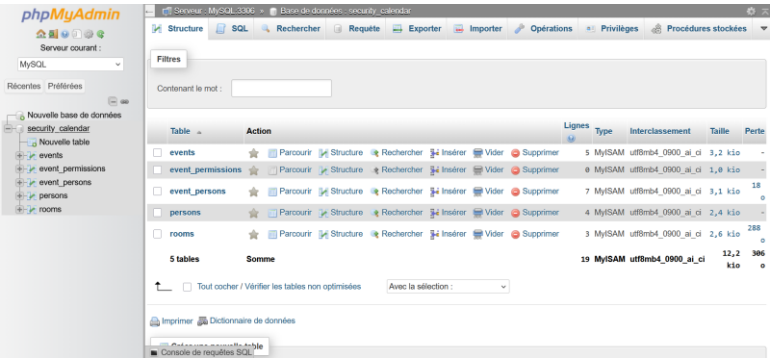
L’objectif principal et donc de créer des évènements, il faut donc pouvoir renseigner des salles et des personnes. Pour cela, je décide de créer une barre latéral me permettant de naviguer rapidement entre les plusieurs données à renseigner. Je créer donc 4 principales pages : Le calendrier, Les personnes, Les salles et Les évènements :

```html
<div id="sidebar">
    <nav>
        <a href="/stage2/events/index.php">📅 Calendrier</a>
        <a href="/stage2/events/list.php">📂 Évènements</a>
        <a href="/stage2/persons/list.php">👤 Personnes</a>
        <a href="/stage2/rooms/list.php">🏢 Salles</a>
    </nav>
</div>

<div id="menuOverlay"></div>
<button id="menuToggle">☰</button>

<script>
document.addEventListener('DOMContentLoaded', () => {
    const btn = document.getElementById('menuToggle');
    const side = document.getElementById('sidebar');
    const over = document.getElementById('menuOverlay');
    const content = document.querySelector('.main-content');

    if (btn && side && over) {
        const toggle = () => {
            side.classList.toggle('open');
            over.classList.toggle('active');
            if (content) content.classList.toggle('shifted');
        };
        btn.onclick = toggle;
        over.onclick = toggle;
    }
});
```



Cependant, pour pouvoir renseigner des personnes, des salles et des évènements, il est nécessaire de créer une **base de données**. J’ai donc utilisé **WAMP** (une plateforme de développement Web permettant de faire fonctionner localement (sans avoir à se connecter à un serveur externe) des scripts PHP) et crée ma base dans « **phpmyadmin** » (logiciel libre écrit en PHP destiné **à gérer l'administration d'un serveur de base de données MySQL ou MariaDB**)

```sql
CREATE DATABASE security_calendar CHARACTER SET utf8mb4;
USE security_calendar;

CREATE TABLE persons (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    prenom VARCHAR(100) NOT NULL,
    secteur VARCHAR(100),
    photo VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE rooms (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom_salle VARCHAR(100) NOT NULL,
    emplacement VARCHAR(150),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE events (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    room_id INT NOT NULL,
    start_datetime DATETIME NOT NULL,
    end_datetime DATETIME NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (room_id) REFERENCES rooms(id)
        ON DELETE CASCADE
);

CREATE TABLE event_persons (
    event_id INT NOT NULL,
    person_id INT NOT NULL,

    PRIMARY KEY (event_id, person_id),

    FOREIGN KEY (event_id) REFERENCES events(id)
        ON DELETE CASCADE,

    FOREIGN KEY (person_id) REFERENCES persons(id)
        ON DELETE CASCADE
);
```



Cela me permet donc d'assurer la **persistance**, **l'organisation** et la **gestion dynamique** des données du calendrier.

Ensuite, j'ai crée les pages permettant de gérer les personnes, les salles et les évènements. J'utilise donc plusieurs dossier et sous-dossier différent pour que chaque informations correspondent à la bonne page. Dans C:/wamp64/www/stage2 on peut retrouver :

Nom	Modifié le
assets	26/01/2026 10:40
calendar	20/01/2026 10:59
config	16/01/2026 11:20
events	29/01/2026 15:41
includes	20/01/2026 15:24
partials	26/01/2026 10:24
persons	30/01/2026 11:04
rooms	30/01/2026 11:19
sql	16/01/2026 11:35
uploads	16/01/2026 14:25
alertes.php	05/02/2026 16:54
db.php	21/01/2026 10:10
index.php	20/01/2026 11:57

Pour la création, la modification et la suppression d'évènement, il faut créer dans chacun des dossier (rooms, persons et event) ces fichiers :

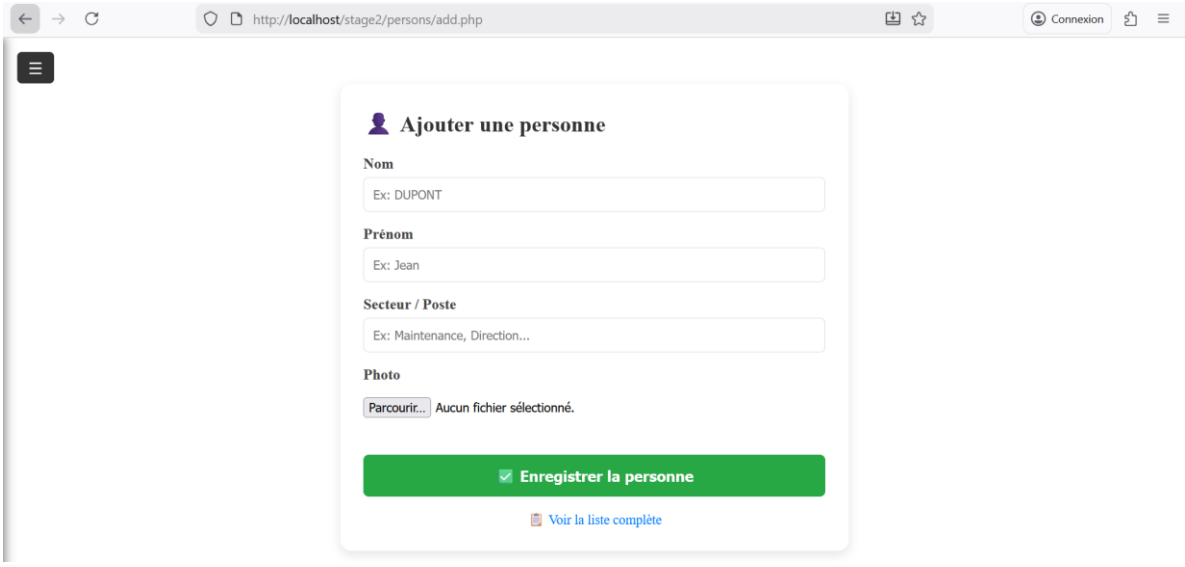
add.php	30/01/2026 10:48
delete.php	30/01/2026 11:44
edit.php	30/01/2026 11:43
list.php	30/01/2026 11:41

Ainsi, grâce à « add.php », je peux ajouter, « delete.php » supprimer, « edit.php » modifier et « list.php » lister les données.

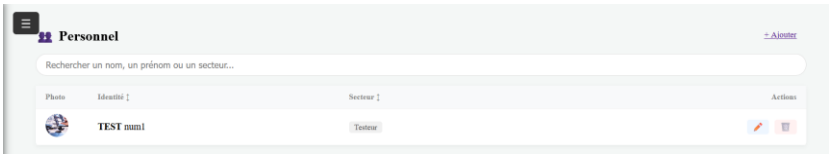
La gestion des personnes :

L'interface de la page « persons » est présenté comme :

En haut à droite, je peux ajouter une personne. Lorsque j'appuie sur le bouton, j'ai la possibilité de créer une personne en fonction de son prénom, son nom, son secteur d'activité et sa photo (nécessaire pour le bon fonctionnement de la détection de visage)



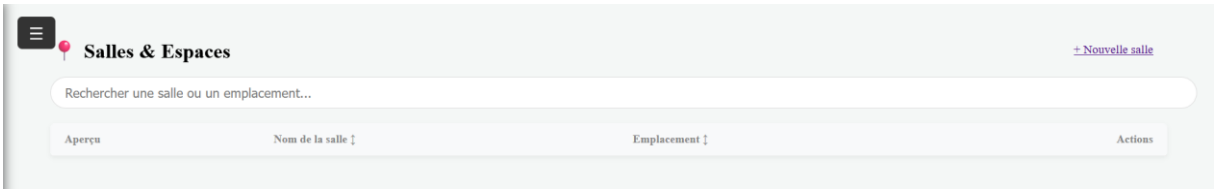
Nous pouvons créer une personne « test » pour vérifier que tout fonctionne bien :



La personne a bien été ajouté. Il est aussi possible de modifier et de supprimer cette personne :

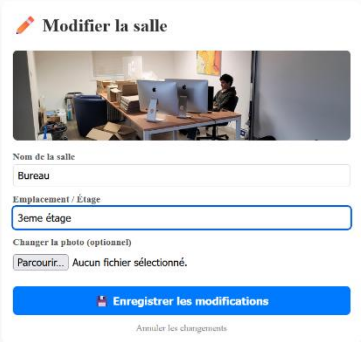
La gestion des salles :

Pour les salles, il s'agit des mêmes manipulations :



Pour ajouter une salle, il faut cliquer sur « + Nouvelle salle » et remplir les informations :

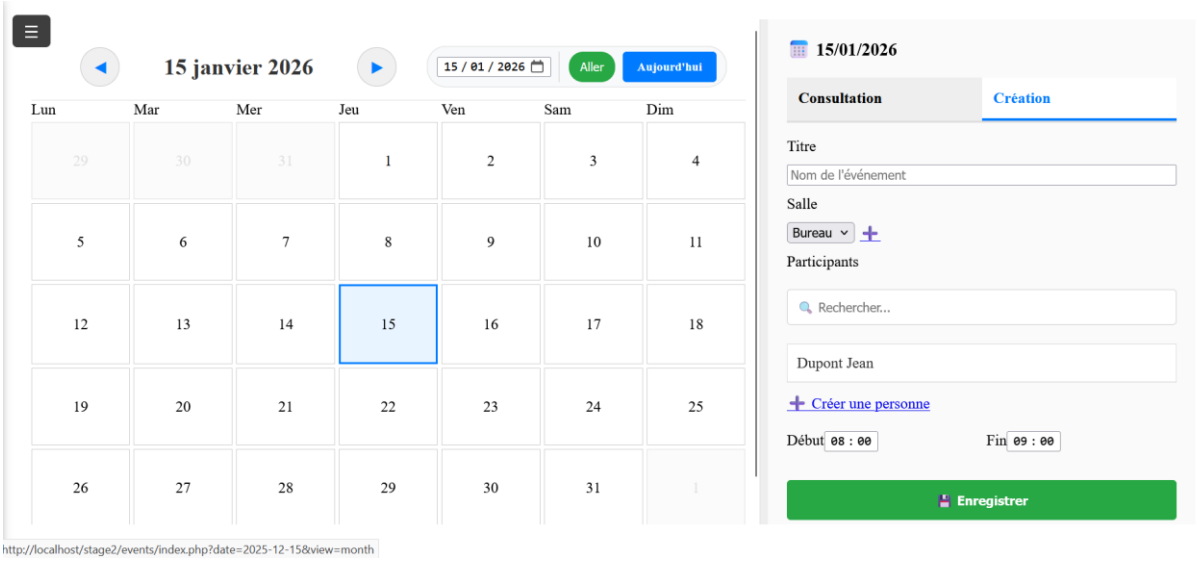
Idem pour la modification et la suppression :



Les évènements :

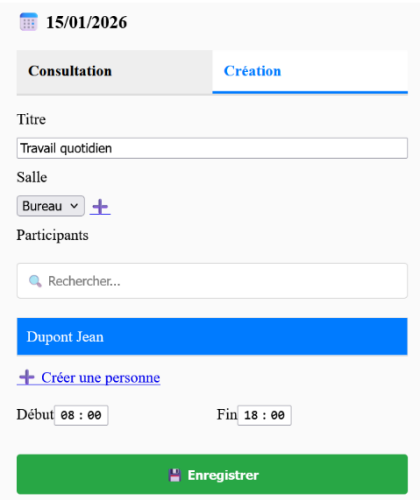
Les évènement aussi ont une page qui permet de recenser l'ensemble des informations comme les dates (jours/horaires) et les personnes et salles habilitées.

Cependant, pour pouvoir directement créer un évènement, il faut se déplacer dans le calendrier. Pour cela, on clique sur le jour concerné par l'évènement et on rempli les informations :

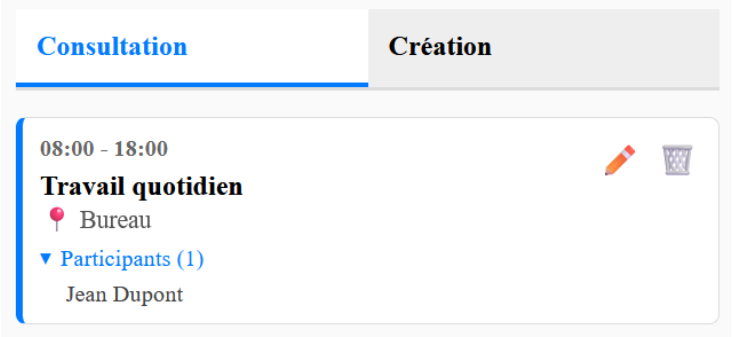


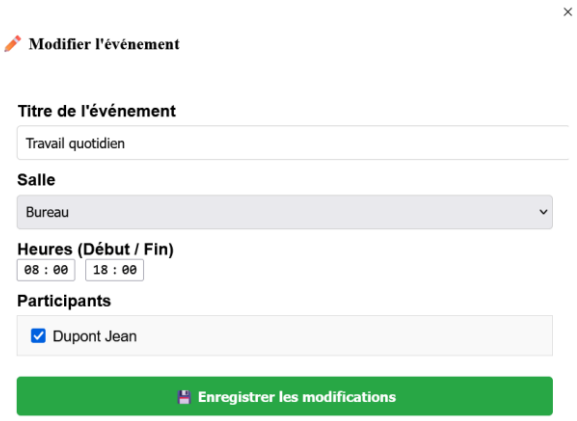
(on peut voir un onglet s'ouvrir permettant de consulter tous les évènements de la journée et un autre permettant de les créer. On peut aussi, depuis cette affichage, créer une personne et une salle.)

Supposons que nous voulons que Dupont Jean n'ai accès qu'à la salle 'Bureau' de 8h00 à 18h00 pour son travail quotidien, nous aurions ceci :



Puis, nous avons la possibilité de consulter l'évènement pour modifier ou supprimer celui-ci :





Le lien entre le calendrier et les caméras :

Le projet initial avait pour objectif une reconnaissance faciale en temps réel avec une analyse en continue du flux vidéo. Cependant, le développement des informations (traitement CPU uniquement, absence d'accélération GPU) ont entraîné une charge du processeur trop importante.

La bibliothèque de reconnaissance faciale utilisée réalise des calculs complexes pour chaque visage détecté, ce qui prend beaucoup de ressources. Lors des analyses continues, de nombreux ralentissement et instabilité du système étaient remarquable.

Pour assurer l'avancement du projet, il a fallu:

- Réduire la résolution d'analyse,
- Limiter la fréquence des opérations de reconnaissance,
- Séparer les phases de détection et de reconnaissance,

Grâce à ces adaptations j'ai pu obtenir un système fonctionnel, stable et cohérent avec les ressources matérielles disponibles.

Dans un premier temps, j'ai décidé de développer à partir de ma webcam locale plutôt qu'un flux RTSP. Ce choix m'a permis de réduire les variables techniques externes (latence réseau, décodage vidéo, instabilité du flux), et de me concentrer principalement sur le pipeline de détection et de reconnaissance faciale.

Cette idée permet aussi d'isoler les problématiques liées aux performances CPU et d'optimiser les traitements avant toute intégration à un flux distant (et donc intégrer le RTSP une fois l'environnement local opérationnelle).

Pour réussir à faire communiquer le calendrier et l'application de détection faciale, il faut comprendre le rôle de chacun.

1- La base de données comme moyen de communication

Côté Web (PHP) : enregistrement de fait, indiquant qui a le droit d'être dans tel lieux et à telle heure

Côté IA (Python) : Lorsqu'un visage est reconnu, le script va interroger la même base de donnée que côté web (la base de données relationnelle **MySQL**)

2- Le processus de vérification

Lorsqu'un visage est détecté, le système suit l'algorithme en 3 étapes qui va permettre de :

1. **Identifier (Reconnaissance Faciale) :** Grâce au module **face_recognition**, un encodage du visage est créé et le compare à la base de données des visages connus
2. **Requête d'autorisation (Vérification du Planning) :** le script python exécute une requête SQL **SELECT** pour vérifier la présence des évènement correspondant comme :
    - **Identifiant de la personne** (suite à la reconnaissance)
    - **Identifiant de la salle** (pour le moment fixé au lancement du script)
    - **Horodatage** (L'heure système **time.time()** doit être compris entre **start_datetime** et **end_datetime**)
3. **Réponse :** Si la requête renvoie un résultat ; l'accès est 'Autorisé' cependant, si la requête est vide, l'accès est 'Refusé' et la procédure d'alerte se déclenche.

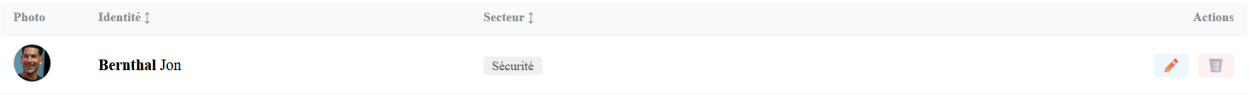
3- Le processus de vérification

Enfin, en cas d'intrusion, le script Python « communique » aussi en retour avec l'interface Web permettant de créer un **Stockage physique** (enregistrement de capture JPEC des intrus dans un répertoire (/alerts)) et d'un **Affichage dynamique** (le fichier PHP du calendrier scanne le répertoire pour afficher les dernières alertes dans le tableau de bord, permettant une surveillance centralisée).

Démonstration complète de la combinaison Calendrier/Détection de visage :

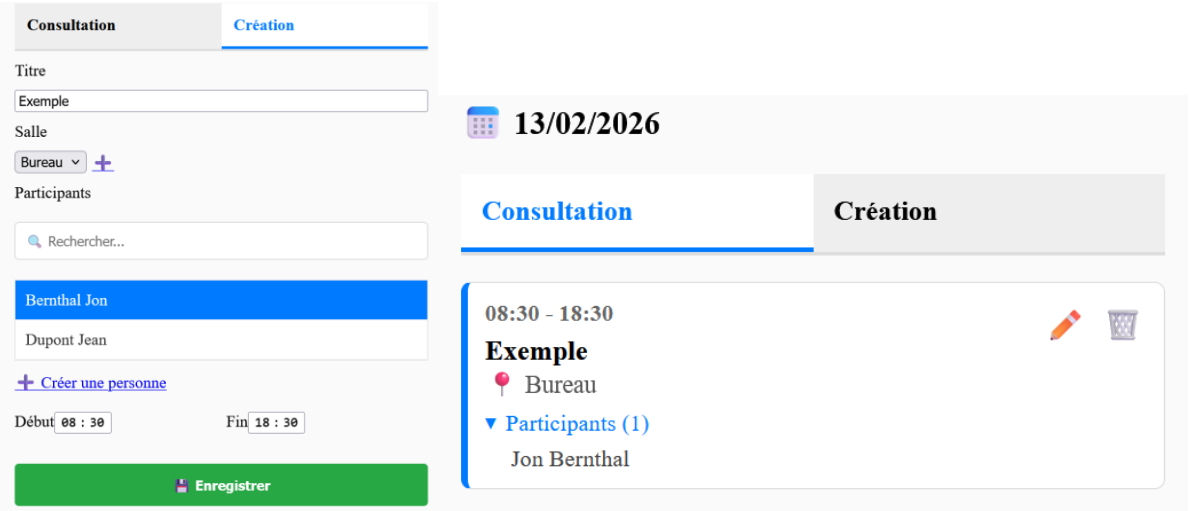
Pour qu'un évènement n'indique aucune erreur, il faut que toutes ses conditions soient respectés (que seuls les personnes autorisées à X salle et à X heure soient détectées).

Créons une personne (ici, nous allons prendre l'acteur Jon Bernthal pour notre exemple) :



Puis une salle :

Enfin, nous créons l'évènement qui donne le droit d'accès aux bureaux du 1er étage à Jon Bernthal de 08h30 à 18h30 pour le vendredi 13 Février 2026:

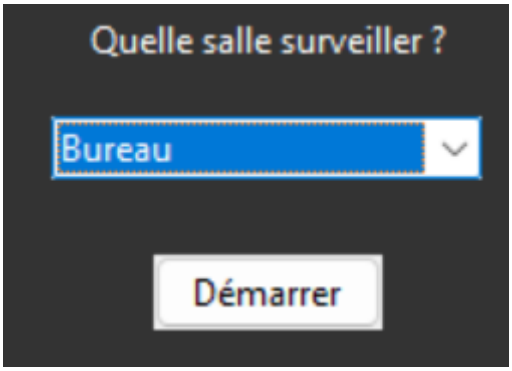


Maintenant que le calendrier contient un évènement, vérifions si les caméras sont capables de reconnaitre Jon et de savoir si il a le droit ou non d'être dans cette zone.

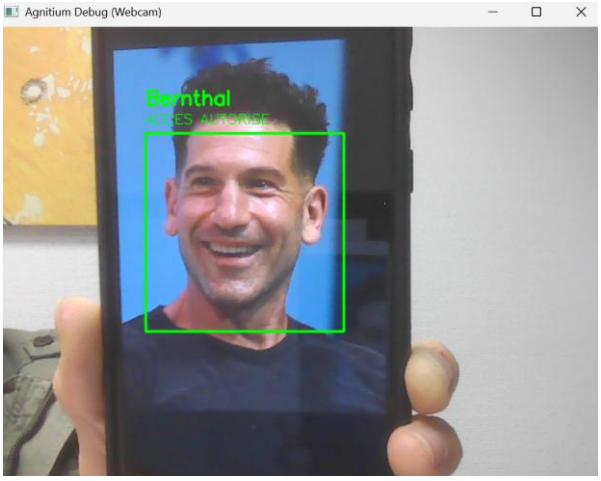
Pour lancer le programme, il faut se rendre dans le dossier du projet caméra « Agnitium » et lancer le menu.py :

```
theot@Asus04 MINGW64 ~
$ cd /c/Users/theot/Agnitium/

theot@Asus04 MINGW64 ~/Agnitium (main)
$ python menu_app.py
```

On choisit la salle à surveiller « Bureau » puisqu'il s'agit de la salle où la validation de l'évènement nous intéresses.

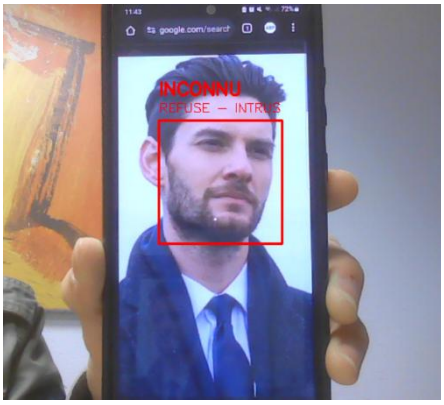


Et on remarque que Bernthal est bien autorisé :



Mais, si je venais à modifier les horaires d'autorisation de la salle pour Bernthal (12h30 – 18h30), il serait signalé « Hors planning » :

Cependant, pour une personne n'ayant aucune autorisation et étant inconnu à la base de données, le message sera encore différent (prenons pour exemple un autre acteur).



L'individu est inconnu, donc toutes autorisations est refusé.

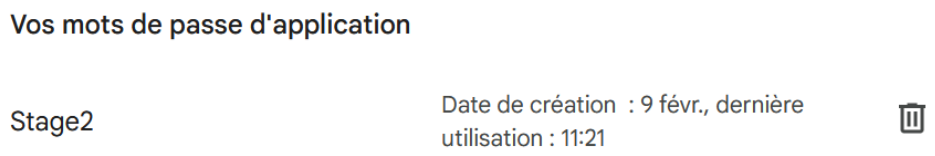
En cas d'intrusion, il est nécessaire de pouvoir être avertis et de garantir une réactivité immédiate. Pour cela, le plus efficace est de développé un système de notification par courrier électronique.

L'envoi des alertes repose sur le protocole **STMP** (**S**imple **M**ail **T**ransfert **P**rotocol) via les serveurs de Google (Gmail).

- Côté **Sécurité**, l'utilisation de **STARTTLS** sécurise la connexion entre le script et le serveur mail.

```python
def send_alert_email(person_name, room_id, image_path=None):
    msg = EmailMessage()
    msg.set_content(f"Alerte : {person_name} a tenté d'accéder à la salle {room_id}.", charset='utf-8')
    msg['Subject'] = f"🚨 INTRUSION - SALLE {room_id}"
    msg['From'] = "theo.trn4@gmail.com"
    msg['To'] = "cyber.test1423@gmail.com"

    # Si une image est fournie, on l'attache au mail
    if image_path and os.path.exists(image_path):
        try:
            with open(image_path, 'rb') as f:
                img_data = f.read()
                msg.add_attachment(img_data, maintype='image', subtype='jpeg', filename=os.path.basename(image_path))
        except Exception as e:
            print(f"Impossible de joindre l'image : {e}")

    try:
        with smtplib.SMTP('smtp.gmail.com', 587) as smtp:
            smtp.starttls()
            smtp.login("theo.trn4@gmail.com", "".replace(" ", ""))
            smtp.send_message(msg)
        print("📧 Mail d'alerte avec photo envoyé !")
    except Exception as e:
        print(f"❌ Erreur mail : {e}")
```

- Côté **Authentification**, pour protéger mon compte personnel, j'ai du utilisé un « **Mot de passe d'application** ». Cela permet notamment au script Python de se connecter sans avoir besoin du mot de passe principal du compte (ce qui permet de renforcer la sécurité de tout le dispositif)

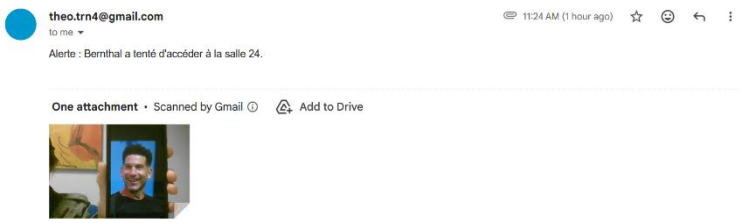


Pour éviter de saturer la boîte mail de l'administrateur, il faut intégrer des variables permettant de contrôler l'envoi des mails comme **EMAIL_COOLDOWN**.

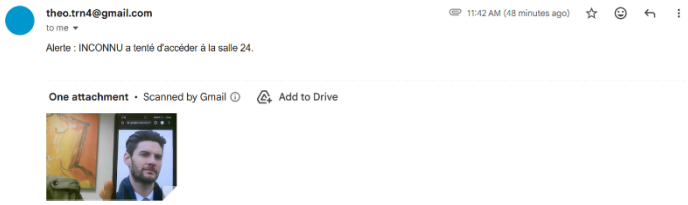
Enfin, pour être encore + réactif et obtenir les informations nécessaire pour obtenir un contexte complet, le mail renvoie les informations suivantes :

- **Identité :** Nom de la personne reconnue (ou « inconnu »)
- **Localisation :** Nom ou ID de la salle concernée
- **Horodatage :** Date et heure précise de l'incident
- **Preuve Visuelle :** Capture d'écran réalisée au moment de la détection est jointe au message en format JPEG

Ainsi, pour les intrusions de toute à l'heure, les mails ont bien été envoyés :



Pour Jon Bernthal, le message est « Alerte : Bernthal a tenté d'accéder à la salle 24. » puisque Jon est connu dans la base de donnée.

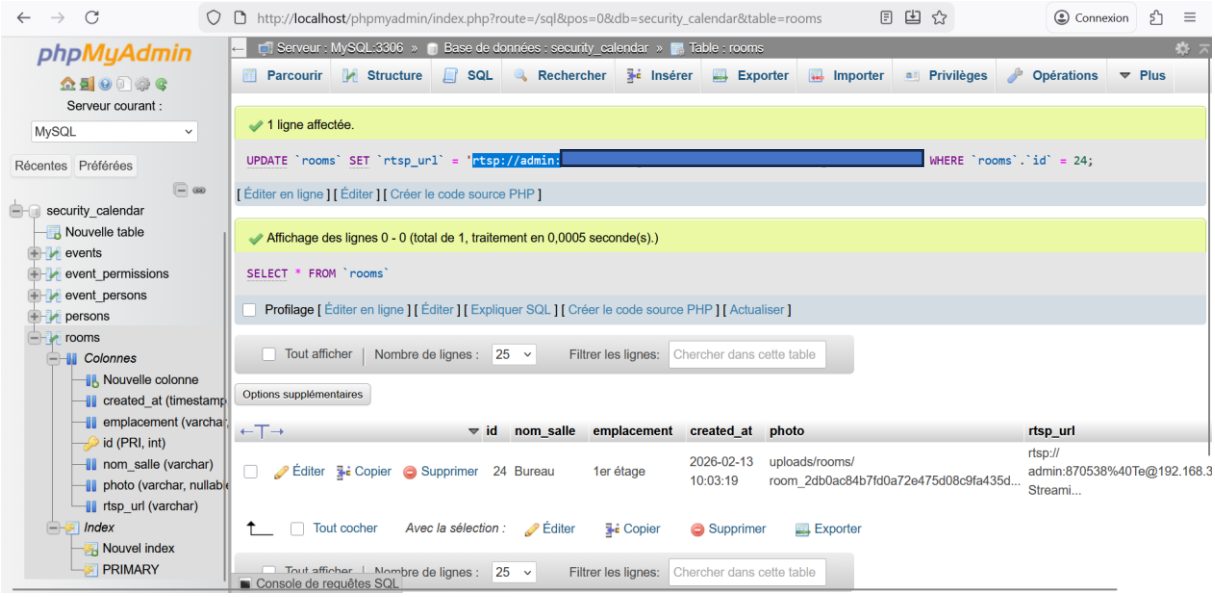


Pour une personne non-connu de la base de donnée, le message est « Alerte : INCONNU a tenté d'accéder à la salle 24. » avec aussi une photo.

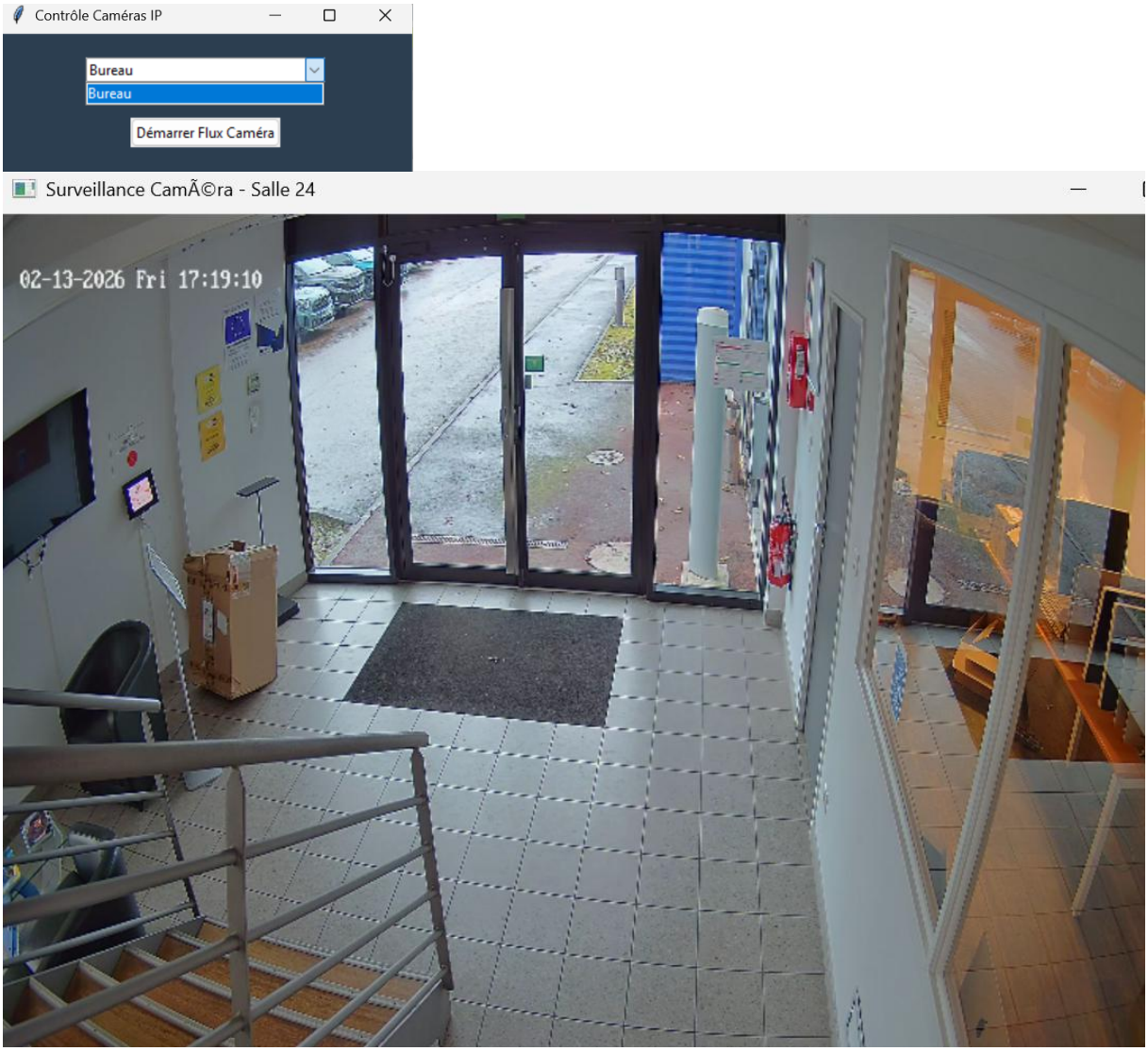
Utilisation Calendrier/Détection de visage avec Caméra RTSP :

Pour un pc puissant, l'algorithme est capable de détecter et scanner un visage rapidement et plusieurs fois par seconde, faire du tracing, analyser jusqu'à 6 visages en permanence.

Pour relier une salle à un URL RTSP, il faut créer une table « rtsp_url » dans la base de données. Puis, ajouter cette URL à la salle souhaité. Dans notre exemple précédant, on ajoute « rtsp://admin:???? » à la salle « bureau ».



Puis, nous pouvons lancer le programme :



Ce projet a permis de concevoir une solution de sécurité innovante dont l'intelligence repose sur la synchronisation en temps réel de deux mondes : la **planification administrative** et la **surveillance biométrique**.

1. Le Calendrier comme Cerveau du Système

La véritable innovation de ce projet ne réside pas uniquement dans la reconnaissance faciale, mais dans sa subordination totale au **calendrier dynamique**. Dans cette architecture, la base de données security_calendar n'est plus un simple outil de consultation, elle devient le référentiel d'autorité. C'est le calendrier qui dicte à l'IA qui a le droit d'être présent, où, et à quel moment précis. Cette approche transforme une surveillance passive en un contrôle d'accès proactif et intelligent.

2. Une Architecture "Data-Driven" (Pilotée par la donnée)

Grâce à l'utilisation du protocole **RTSP** et de la base de données MySQL, nous avons créé un système entièrement piloté par les données :

- **Flexibilité Temporelle :** Toute modification dans le calendrier PHP est instantanément répercutée sur le moteur de détection Python.
- **Flexibilité Matérielle :** Les adresses IP des caméras sont gérées en base de données, permettant une maintenance et une évolution du parc matériel sans aucune modification du code source.

3. Un Pont entre Gestion et Sécurité

Le développement de cette solution hybride démontre que les outils de gestion d'entreprise (ERP, Planning, SIRH) peuvent être couplés à des technologies de vision par ordinateur pour renforcer la sécurité des infrastructures. Sur un matériel de production haute performance, ce système assure une vérification de conformité entre le "théorique" (le planning prévu) et le "réel" (les flux vidéo en direct), garantissant ainsi une gestion des accès sans faille.

4. Bilan du Stage et Perspectives

Ce projet m'a permis de maîtriser l'intégralité de la chaîne de traitement de l'information : de la saisie d'un événement en PHP à la détection faciale sous Python, jusqu'à l'alerte SMTP automatisée. Cette expérience confirme l'importance de l'interopérabilité des systèmes dans le développement logiciel moderne. À l'avenir, une telle solution pourrait être étendue à une gestion de présence automatisée, où le calendrier se remplirait de lui-même en fonction des détections réelles, bouclant ainsi le cycle de l'automatisation.